# SESIÓN DE CLASE — Django REST Framework + API (FastAPI) + Proyecto Integrador

*Sesión N° 4 — 8 Octubre de 2026 (Jueves) — Bloque 1: Django REST Framework*

| | | | |
|---|---|---|---|
| **Instructor** | César Augusto Moreno Mena | **Centro** | CESGE |
| **Curso** | Django REST Framework + API (FastAPI) + Proyecto Integrador | **Año** | 2026 |

| N° | Fecha | Día | Semana | Bloque | Tema de la sesión | Teoría | Ejemplo | Práctica |
|---|---|---|---|---|---|---|---|---|
| 4 | 8 Octubre | Jueves | S2 | Bloque 1: Django REST Framework | ***Repaso y practica integradora*** | Sesion de consolidacion de los temas vistos en las sesiones anteriores de Django REST Framework: Introducción a Django, Modelos y ORM de Django, Django Admin. Aclaracion de dudas y retroalimentacion grupal sobre los avances del proyecto. | Revision guiada en vivo de los ejercicios y avances entregados por los aprendices, con correccion de errores comunes relacionados con: Introducción a Django, Modelos y ORM de Django, Django Admin. | Los aprendices resuelven ejercicios adicionales de refuerzo y avanzan en su proyecto personal aplicando lo trabajado en: Introducción a Django, Modelos y ORM de Django, Django Admin. |

## 1. Objetivo de la sesión

Esta es una sesión de **repaso y práctica integradora**. El objetivo es que el aprendiz **consolide** en un solo proyecto completo los tres temas vistos este bloque: *Introducción a Django* (2 de octubre), *Modelos y ORM* (5 de octubre) y *Django Admin* (7 de octubre), resolviendo dudas y corrigiendo errores comunes antes de avanzar a Django REST Framework.

## 2. Repaso rápido de conceptos clave

### Sesión 1 (2 oct) — Introducción a Django

| | |
|---|---|
| ✓ | Django es un framework "con baterías incluidas" que organiza el código con el patrón **MTV** (Model – Template – View). |
| ✓ | Un **proyecto** se crea con `django-admin startproject` ; una **app**, con `python manage.py startapp` . |
| ✓ | Toda app nueva debe registrarse en `INSTALLED_APPS` (en `settings.py` ). |
| ✓ | Una vista se conecta a una URL mediante `urls.py` (el de la app, incluido en el del proyecto con `include()` ). |

### Sesión 2 (5 oct) — Modelos y ORM

| | |
|---|---|
| ✓ | Un **modelo** es una clase de Python que representa una tabla; cada atributo es una columna. |
| ✓ | El **ORM** traduce Python a SQL ( `.all()` , `.filter()` , `.get()` , `.create()` ). |
| ✓ | Todo cambio en `models.py` requiere `makemigrations` **y luego** `migrate` . |
| ✓ | Usa `DecimalField` para dinero, nunca `FloatField` . |

### Sesión 3 (7 oct) — Django Admin

| | |
|---|---|
| ✓ | El Admin se activa creando un superusuario con `python manage.py createsuperuser` . |
| ✓ | Un modelo se registra con `@admin.register(Modelo)` sobre una clase que hereda de `admin.ModelAdmin` . |
| ✓ | `list_display` muestra columnas, `list_filter` agrega filtros, `search_fields` agrega búsqueda. |

## 3. Autoevaluación: ¿qué tan seguro te sientes?

Antes de empezar el ejercicio integrador, responde honestamente estas preguntas (puedes comentarlas con tu instructor si tienes dudas en alguna):

1. ¿Puedo explicar con mis propias palabras qué hace cada letra de **MTV**?
2. ¿Sé la diferencia entre `makemigrations` y `migrate` ?
3. ¿Sé cuándo usar `ForeignKey` y qué hace `on_delete` ?
4. ¿Sé registrar un modelo en el Admin y personalizar su listado?
5. Si mi servidor da un error "Page not found (404)", ¿sé en qué archivos debo revisar primero?

*Si respondiste "no" a alguna pregunta, aprovecha el espacio de dudas de hoy antes de avanzar al ejercicio integrador.*

## 4. Ejercicio integrador paso a paso

Vamos a construir, de principio a fin, un pequeño sistema de **"Biblioteca"** que combina los tres temas del bloque. Sigue los pasos en orden:

### Paso 1: Proyecto y app (repaso sesión 1)

```
django-admin startproject config .
python manage.py startapp biblioteca
```

Registra `biblioteca` en `INSTALLED_APPS`.

### Paso 2: Modelos (repaso sesión 2)

En `biblioteca/models.py`, crea:

```
from django.db import models

class Autor(models.Model):
    nombre = models.CharField(max_length=120)
    nacionalidad = models.CharField(max_length=60, blank=True)

    def __str__(self):
        return self.nombre


class Libro(models.Model):
    titulo = models.CharField(max_length=200)
    autor = models.ForeignKey(Autor, on_delete=models.CASCADE, related_name='libros')
    precio = models.DecimalField(max_digits=8, decimal_places=2)
    disponible = models.BooleanField(default=True)
    fecha_publicacion = models.DateField(null=True, blank=True)

    def __str__(self):
        return self.titulo
```

```
python manage.py makemigrations
python manage.py migrate
```

### Paso 3: Datos de prueba desde el shell (repaso sesión 2)

```
python manage.py shell
```

```
>>> from biblioteca.models import Autor, Libro
>>> a1 = Autor.objects.create(nombre="Gabriel García Márquez",
nacionalidad="Colombia")
>>> a2 = Autor.objects.create(nombre="Isabel Allende", nacionalidad="Chile")
>>> Libro.objects.create(titulo="Cien años de soledad", autor=a1, precio=68000)
>>> Libro.objects.create(titulo="El amor en los tiempos del cólera", autor=a1,
precio=55000)
>>> Libro.objects.create(titulo="La casa de los espíritus", autor=a2, precio=60000)
```

### Paso 4: Admin personalizado (repaso sesión 3)

En `biblioteca/admin.py`:

```
from django.contrib import admin
from .models import Autor, Libro

@admin.register(Autor)
class AutorAdmin(admin.ModelAdmin):
    list_display = ('nombre', 'nacionalidad')
    search_fields = ('nombre',)


@admin.register(Libro)
class LibroAdmin(admin.ModelAdmin):
    list_display = ('titulo', 'autor', 'precio', 'disponible')
    list_filter = ('disponible', 'autor')
    search_fields = ('titulo',)
```

```
python manage.py createsuperuser
python manage.py runserver
```

Entra a `/admin/`, verifica que los 3 libros y 2 autores aparezcan correctamente, que el filtro por autor funcione y que la búsqueda por título funcione.

### Paso 5: Vista simple final (repaso sesión 1)

Como cierre, crea una vista en `views.py` que devuelva cuántos libros hay registrados:

```
from django.http import HttpResponse
from .models import Libro

def resumen(request):
    total = Libro.objects.count()
    return HttpResponse(f"Actualmente hay {total} libros registrados en la
biblioteca.")
```

Conéctala a la ruta `/biblioteca/resumen/` siguiendo el mismo procedimiento de la sesión 1 (archivo `urls.py` de la app + `include()` en el del proyecto).

## 5. Errores comunes del bloque (repaso general)

| | |
|---|---|
| ✗ | Olvidar registrar la app en `INSTALLED_APPS` → la app "no existe" para Django. |
| ✗ | Modificar `models.py` y olvidar `makemigrations` + `migrate` → error "no such table" o datos desactualizados. |
| ✗ | Olvidar `on_delete` en un `ForeignKey` → error al migrar. |
| ✗ | No registrar el modelo en `admin.py` → no aparece en el panel `/admin/`. |
| ✗ | Falta el método `__str__` → los registros se ven como `Objeto object (1)` en vez de un nombre legible. |

## 6. Espacio de dudas y retroalimentación

Este es el momento de la sesión para plantear al instructor cualquier duda pendiente del bloque: errores de instalación, mensajes de error que no entendiste, o partes de la arquitectura MTV que no quedaron claras. Trae tu proyecto personal abierto para revisión en vivo.

## 7. Checklist final del bloque (Sesiones 1–3)

| | |
|---|---|
| ☐ | Mi proyecto Django corre sin errores con `python manage.py runserver`. |
| ☐ | Tengo al menos 2 modelos relacionados con `ForeignKey`. |
| ☐ | Mis migraciones están aplicadas sin errores. |
| ☐ | Puedo crear y consultar datos desde el shell de Django. |
| ☐ | Mi panel de administración está personalizado con `list_display`, `list_filter` y `search_fields`. |

*Próxima sesión (9 de octubre): iniciamos Introducción a Django REST Framework, donde convertiremos estos mismos modelos en una API que entrega datos en formato JSON.*